



Trabajo Práctico: Concurrency, Channels

y Crítica de Sistemas (Go)

Materia: Programación Concurrente

Ciclo Lectivo: 2026

Introducción y Objetivos

El presente trabajo práctico tiene como finalidad profundizar en el modelo de concurrencia de Go (*CSP - Communicating Sequential Processes*). Se evaluará no solo la capacidad de escribir código que funcione, sino la habilidad para detectar bloqueos (*deadlocks*), gestionar el ciclo de vida de las goroutines y mantener una postura crítica frente a soluciones automatizadas (IA).

Parte I: El Acertijo de los Canales (Comprensión)

Analicen el siguiente fragmento de código que intenta enviar mensajes de log a un sistema de monitoreo.

```
package main
import "fmt"

func main() {
    ch := make(chan string, 2)
    ch <- "Log 1: Sistema iniciado"
    ch <- "Log 2: Conexión establecida"
    ch <- "Log 3: Error de red detectado" // ¿Qué sucede en esta
    línea?

    fmt.Println(<-ch)
    fmt.Println(<-ch)
}
```

Consignas:

1. Al ejecutar este código, el programa falla. Expliquen técnicamente **por qué ocurre un "deadlock"** en la tercera inserción al canal.

Ocurre un deadlock porque el canal con buffer tiene una capacidad limitada de almacenamiento interno, en este caso de 2, al intentar enviar un tercer elemento a un canal cuyo buffer está lleno, el comportamiento nativo de Go determina que la go:routine actual debe bloquearse hasta que otra operación libere el espacio en el canal mediante una lectura. Las líneas encargadas de liberar el canal (`fmt.Println(<-ch)`) se encuentran después de la línea que intenta insertar el "Log 3".

Como la go:routine main se bloquea en la línea del "Log 3", el flujo del programa queda pausado y nunca llega a las instrucciones de lectura.

2. Si eliminamos el número 2 en la creación del canal (`make(chan string)`), el programa

falla mucho antes. Expliquen el concepto de **Rendezvous** (encuentro) y por qué es fundamental para entender los canales sin buffer.

Al eliminar el número 2, se transforma en un canal sin buffer. En programación concurrente, el Rendezvous es un mecanismo de sincronización estricta donde dos procesos independientes (en este caso, dos goroutines) deben encontrarse en el mismo instante de tiempo para poder intercambiar información. Un canal sin buffer tiene una capacidad de almacenamiento de cero. No tiene memoria interna para guardar mensajes, por lo que exige un Rendezvous obligatorio:

- **El Emisor se bloquea** hasta que un Receptor esté listo para leer.
- **El Receptor se bloquea** hasta que un Emisor esté listo para enviar.

Por qué falla el código: Cómo se intenta entregar el "Log 1", el canal no tiene memoria, queda congelado esperando que alguien lo tome. Ya que el código es puramente secuencial y no se crea ninguna otra goroutine en paralelo, no hay nadie para recibir el mensaje. Al bloquearse en la línea del "Log 1", el programa ni siquiera llega a procesar las líneas siguientes ("Log 2" o "Log 3"). El *runtime* de Go detecta que la única go:routine viva se ha quedado en sleep esperando el encuentro, y lanza el error de deadlock en el acto.

Parte II: El "Bug del Principiante" (Debugging y Sincronización)

Un desarrollador junior entregó el siguiente código para procesar 5 tareas en paralelo. El código compila, pero al ejecutarlo se produce un error crítico de ejecución.

Go

```

func worker(id int, resultados chan string, wg *sync.WaitGroup) {
    defer wg.Done()
    // Simula una tarea pesada
    resultados <- fmt.Sprintf("Resultado de tarea %d", id)
}

func main() {
    resultados := make(chan string)
    var wg sync.WaitGroup

    for i := 1; i <= 5; i++ {
        wg.Add(1)
        go worker(i, resultados, &wg)
    }

    wg.Wait()      // Línea de conflicto A
    close(resultados)

    for res := range resultados {
        fmt.Println(res)
    }
}

```

Consignas:

1. Identifiquen el **Deadlock Circular**: Expliquen por qué el `main` queda bloqueado en `wg.Wait()` mientras los `workers` quedan bloqueados intentando enviar datos al canal `resultados`.

Los canales sin buffer en go no pueden guardar datos, los canales unbuffered no poseen una memoria intermedia por la cual guardar el pasaje de datos para posteriormente utilizarlos en la lectura del canal, por ende, cada vez que se quiere insertar un dato al canal, tiene que haber alguien que lo consuma, si no hay alguien que lo consuma, el programa se queda esperando a que suceda ello. Justamente el código expuesto arriba es un ejemplo de esto, el programa declara el canal unbuffered, y manda concurrentemente a hilos que inserten texto al canal, el problema es que los hilos quedan en la inserción y nunca ejecutan el “defer wg.Done()” ya que se quedan pausados antes esperando a que alguien lea los datos que quieren mandar, ya que el canal no los puede almacenar momentáneamente. Finalmente al ejecutar “wg.Wait()” el programa queda totalmente bloqueado ya que queda a la espera de la finalización de los hilos, los cuales, no pueden terminar hasta que alguien no lea/reciba los datos que quieren enviar

2. Propongan una solución técnica (por ejemplo, el uso de una go:routine adicional para el cierre del canal o el uso de buffers) y **justifiquen por qué su solución elimina el bloqueo**.

La solución más sencilla a este problema es haciendo que el canal pase de ser unbuffered a ser un canal buffered donde se puede guardar esta memoria intermedia y que los hilos no queden

bloqueados por el envío del canal. Pero esta solución es en parte una mala práctica, ya que si sobrepasamos el buffer permitido el programa entraría en modo panic. Por ende propongo el siguiente código que soluciona este error (el código añade una única función concurrente que lee los resultados de los hilos permitiendo que los hilos continúen y logren llegar a él defer wg.Done(). Haciendo que el hilo principal pueda seguir):

```
func lectura(resultados chan string){
    for res := range resultados{
        fmt.Println(res)
    }
}

func main(){
    resultados := make(chan string)
    var wg sync.WaitGroup

    //Nueva función de lectura añadida antes del wait concurrente
    go lectura(resultados)

    for(i := 1; i<=5; i++){
        wg.Add(1)
        go worker(i, resultados, &wg)
    }

    wg.Wait()
    close(resultados)
}
```

Parte III: Generación del Código Base con IA

Utilizar IA de su elección con el siguiente prompt exacto y adjuntar el código generado en su entrega:

*"Actúa como un programador Junior en Go. Diseña un sistema de alta disponibilidad para la aprobación y análisis de fraude de transacciones financieras concurrentes. Una función debe cargar un lote secuencial de exactamente 50 transacciones enviándolas a un canal sin buffer llamado **transacciones_entrantes**. Instancia exactamente 5 goroutines de validación (workers) en paralelo que consuman de ahí, tardando de forma aleatoria entre 100ms y 400ms por operación. Los workers deben enviar el string 'Transacción X aprobada por el Nodo Y' a un canal llamado **registro_contable**. El main debe imprimir los resultados y finalizar de forma limpia usando sync.WaitGroup y close() sin usar bloqueos mágicos de tiempo como time.Sleep."*

Parte IV: Informe de Auditoría y Resiliencia

Una vez obtenido el código, responder:

1. **Análisis de Buffering y Filosofía CSP:** Al ser transacciones financieras en tiempo real, el enunciado exige un canal Unbuffered. Expliquen detalladamente qué propiedad de sincronización matemática (Rendezvous) se perdería si la IA hubiese ignorado el prompt y configurado un canal con buffer.

Al ser un canal unbuffered no se pierde el hecho de que cada vez que se envíe algo, tiene que haber alguien que lo reciba, si no lo hay, el que lo envía tiene que esperar. Si la IA hubiese ignorado el prompt y sabiendo que son transacciones financieras. Podrían haber transacciones financieras encoladas en un buffer y no sabríamos con exactitud en qué momento se realiza la transacción. La sincronización matemática *Rendezvous* nos permite asegurarnos que la transacciones se realicen en tiempo real y que si no hay alguien que los lea sea imposible de realizar la transacción llegando a un deadlock

2. **Trampa del Ciclo de Vida:** Las IA suelen cometer el error de colocar la función **close()** dentro del bucle de los workers o llamar a **close()** sobre un canal mientras otras Goroutines siguen intentando escribir en él. Analicen minuciosamente el código entregado: ¿Sufre el programa del error crítico de lanzar un Panic por escribir en un canal cerrado? Justifiquen analizando la matriz de estados vista en clase.

No, no sufre de panic al intentar escribir en un canal cerrado. En GO, el panic por canal cerrado ocurre únicamente al escribir en un canal ya cerrado. TransaccionesEntrantes lo cierra cargarTransacciones después de enviar las 50 transacciones. Los workers sólo leen de él con range, que termina limpiamente cuando el canal se cierra. Nadie escribe después del close.

En registroContable los workers escriben en él, terminan de escribir y llaman Done, y recién

ahí entonces la go:routine coordinadora cierra el canal, haciendo imposible que un worker escriba después del close.

El peligro aparecería si por ejemplo se llamará close desde el mismo lugar que escribe, o si hubiera muchas goroutines cerrando el mismo canal, que también causa panic. Pero ninguno de los dos casos ocurre en el código dado por la IA.

3. **Refactorización de Resiliencia (Inyección de Select):** *Las redes bancarias fallan. Refacturen el código de la IA agregando un **select** con **time.After** dentro del bloque de los workers. Si la consulta simulada tarda más de 250ms, el worker debe abortar la operación, enviar el mensaje "Transacción X rechazada por Timeout de Red" al canal de resultados y continuar con la siguiente transacción.*

–Código adjunto en la carpeta como “codigo_refactorizado”. Cabe resaltar el hecho que corregimos manualmente caracteres que puso la IA que hacían que el código no compile por un problema de overflow en los bits de string, ya que utilizaba caracteres no definidos en utf

Entregables

En una carpeta:

1. Código fuente en Go (.go).
2. Informe en PDF con las respuestas de las Partes I, II y IV.
3. Prompts utilizados para la IA en formato PDF, pueden compartir el chat completo o lo que ustedes fueron enviando para llegar al resultado final del TP.